

Wolt: how does weather impact delivery times?

Group 25

Anh Pham, Kaapo Lahtinen, Vili Mustakallio,
Tomi Seppälä and Maksim Somelar

Table of contents

1. Introduction

2. Problem formulation

3. Data & cleaning

4. Methods

5. Results & Discussion

6. Conclusion



1. Introduction: What is Wolt?

- Wolt is a delivery firm for food and merchandise. ^[1]
- It was founded in 2014 in Helsinki, Finland. ^[2]
- Made their first delivery the following year. ^[2]
- In 2016 started their worldwide expansion. ^[2]
- In 2022 Wolt merged with US delivery company giant Doordash. ^[1]
- Nowadays, the brand operates in more than 25 countries and has over 10 000 employees. ^[2]



- Wolt's aim is to deliver products (e.g. food from restaurants) to its customers and gain a commission and/or delivery fee by doing it. ^[2]
- There are three main areas that Wolt primarily focuses on and identifies: technology, efficiency and customer experience.
 - Provide technology (i.e. app or website) to deliver products to customers as quickly and precisely as possible.
 - Do it with as a low cost as possible.
 - To ultimately enhance the customer experience. ^[2]

2. Problem formulation

- Wolt tries to optimize its operations constantly. ^[2]
- Our **hypothesis** is that bad weather might have a negative effect on delivery times.
- Therefore, we started to analyze does this happen to Wolt and how could they respond to this.

Research question: **how does weather impact delivery times?**

- **Secondary aims** of the study: How useful it is to use weather data when predicting the delivery times? How strong these impacts are?
- **Motivation:**
 - Accurately predicting delivery times under varying weather conditions would enable the company to optimize its resources and better manage operations.
 - This would enhance the three goals: enhance the technology (i.e. provide better forecasting with ML - models), do the operation more efficiently (i.e. estimate more accurately) and create customer satisfaction.

The formula used by the MinMaxScaler is:

$$\text{Scaled value} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

where $X_{\min}$ and $X_{\max}$ are the minimum and maximum values of the feature, respectively.

3. Data & Cleaning (1/2)

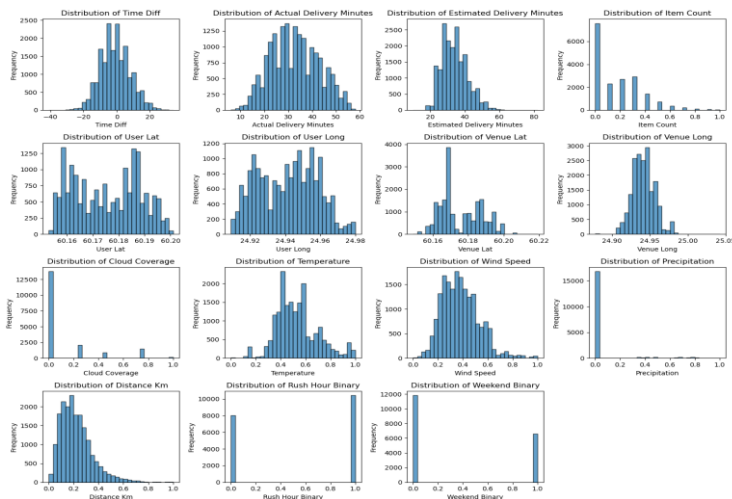
- Wolt published data of autumn 2020 along with its data science internship search in 2024. [3]
 - Called 'orders_autumn_2020.csv'
 - Originally: 12 features (e.g. item_count, wind_speed, temperature, etc.).
 - **Target variable:** actual delivery time.
- We started the data cleaning and modifying process by:
 - Missing values handling:
 - Identified and removed 277 rows from cloud_coverage, wind_speed and temperature.
 - Ensuring that we don't use NaN or null values in our analysis.
 - Feature engineering:
 - Using the Haversine formula we calculated distance.
 - Between user (i.e. user_long, user_lat) and venue (i.e. venue_long, venue_lat) .
 - Created a new feature (i.e. Distance_km)
 - Formatted feature timestamp to datetime and timezone to Helsinki UTC+2.
 - Used MinMaxScaler to item_count, temperature, wind_speed, cloud_coverage, precipitation and distance_km to binary format
 - This way we can normalize & standardize the scale, ensuring the features contribute equally.
 - We categorized rush hours and weekends into binary format.

Continues on the next slide

3. Data & Cleaning (2/2)

- **After the data cleaning** we had 18 429 entries and 13 feature variables in total which of whom we are going to use 8: distance_km, wind_speed, temperature, item_count, cloud_coverage, precipitation, weekend_binary and rush_hour_binary
 - Coordinate data is not necessary, and we come back to Wolt's own estimates later.

Distributions after data cleaning



Data before modifications

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	TIMESTAMP	5 non-null	object
1	ACTUAL_DELIVERY_MINUTES - ESTIMATED_DELIVERY_MINUTES	5 non-null	int64
2	ITEM_COUNT	5 non-null	int64
3	USER_LAT	5 non-null	float64
4	USER_LONG	5 non-null	float64
5	VENUE_LAT	5 non-null	float64
6	VENUE_LONG	5 non-null	float64
7	ESTIMATED_DELIVERY_MINUTES	5 non-null	int64
8	ACTUAL_DELIVERY_MINUTES	5 non-null	int64
9	CLOUD_COVERAGE	5 non-null	float64
10	TEMPERATURE	5 non-null	float64
11	WIND_SPEED	5 non-null	float64
12	PRECIPITATION	5 non-null	float64

dtypes: float64(8), int64(4), object(1)
memory usage: 648.0+ bytes

Data after modification with description of each feature/target variable

1. **ACTUAL_DELIVERY_MINUTES** : The actual delivery time.
2. **ITEM_COUNT** : The number of items in the order.
3. **USER_LAT** : The latitude of the user's location.
4. **USER_LONG** : The longitude of the user's location.
5. **VENUE_LAT** : The latitude of the venue location.
6. **VENUE_LONG** : The longitude of the venue location.
7. **ESTIMATED_DELIVERY_MINUTES** : The base estimated delivery time.
8. **TIME_DIFF** : The difference between the actual and estimated delivery time (negative if delivered earlier, positive if delivered later).
9. **CLOUD_COVERAGE** : Cloud coverage in the area.
10. **TEMPERATURE** : Temperature in the area.
11. **WIND_SPEED** : Wind speed in the area.
12. **PRECIPITATION** : Precipitation in the area.
13. **DISTANCE_KM** : A newly engineered feature representing the distance between the user and the venue (in kilometers).
14. **RUSH_HOUR_BINARY** : Binary variable where 1 indicates rush hours (Lunch: 11 AM to 2 PM, Dinner: 6 PM to 10 PM).
15. **WEEKEND_BINARY** : Binary variable where 1 indicates the order was placed on a weekend.

4. Methods (1/7)

1. Linear regression

- Provides a simple and easy-to-read model that can be used as a baseline for more complex methods.
 - Easy to interpret as it provides straightforward coefficients that tell us how much each weather variable affects delivery time. Helps to understand which factors have significant effect.
 - Computationally efficient and easy to implement.

2. Random Forest

- Captures more complex, non-linear interactions between variables and help identify which variables have the most significant effect.
 - Weather conditions may have a non-linear relationship with delivery times. For example, light drizzle might have no effect on delivery times while heavy rain might have a significant effect.
 - Weather variables have different feature importance and random forest helps identify this. For example, traffic might have more effect on the delivery times than wind.

3. XGBoost

- The random forest model seemed to give contradictive and bad results, thus incorporating XGBoost into the model was deemed appropriate.
 - Ensemble method which builds a series of models sequentially trying to correct the errors of previous ones. This results in a more accurate results.
 - Excellent at capturing non-linear relationships between features.

4. Methods (2/7)

- Ridge and Lasso regression are able to handle datasets that include multicollinearity between variables. Given the nature of our data, which includes multiple factors like weather conditions, travel and rush hour, these methods prevent the model from being too complex or over-fitting.

4. Ridge regression

- Minimizes the potential overfitting that can occur if the model overfits the training data. This feature is useful in our case, where multiple correlated variables can lead to overfitting.
- Ridge regression introduces a little bit of bias which can reduce variance and lead to a better performance on new data.

5. Lasso regression

- Lasso regression reduces some coefficients to zero, leaving out less important variables. It therefore helps identify which factors are truly influential and which are not, making the model more accurate in terms of prediction accuracy and explainability.
- Since some coefficients are set to zero, it creates a simpler and more interpretable model. This helps explain which weather conditions have the most significant impact on delivery times.

4. Methods (3/7) – Features

- **Recursive feature elimination with cross-validation** to select features (RFECV) was used for all models.
 - Applying RFECV to select features for all of the used models ensures that each model is trained on the same optimal subset of features. This consistency is important for making fair comparisons between the models.
 - Model agnosticity of RFECV makes it applicable to any type of model to determine the best features. This ensures that feature selection is handled consistently across all models, and that the comparison between them is valid.
 - Different models have different capacities for handling large feature sets and interactions between variables. RFECV helps ensure that each model is working with the right set of features, preventing unnecessary complexity in simpler models like linear regression, while still giving more powerful models like XGBoost the flexibility to exploit important features.
- In four of the models, after feature selection, all of the 8 hand-selected features remained.
 - However, **with Lasso Regression, the features were eliminated to 5:** *item_count*, *temperature*, *precipitation*, *distance_km* and *rush_hour_binary*.

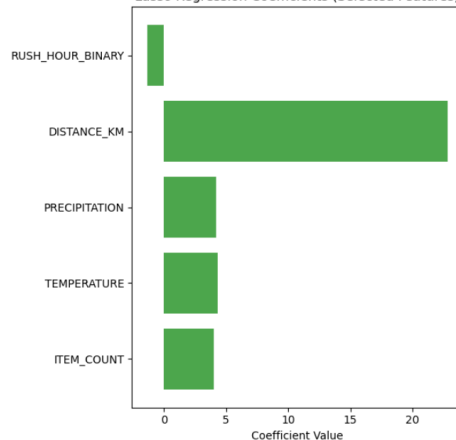
4. Methods (4/7) – Features

- Before moving to the results themselves, we want to highlight some of the minor differences between the models.
- To compare the models of Ridge and Lasso regression together, we can look at the coefficients.
 - Distance has the highest increasing impact on delivery time.
 - Weekends and rush hour seem to lower the delivery time.
 - Lasso and ridge have pretty much the same coefficient values.
 - This is reasonable since they differ on the penalty, but otherwise are pretty similar to each other.
- Linear regression seems to have similar results.

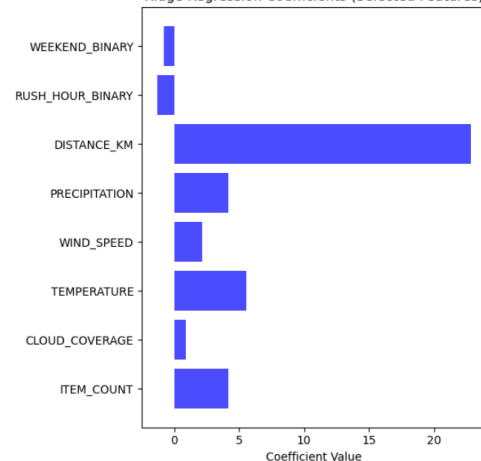
Linear regression coefficients

	coef
const	23.7927
ITEM_COUNT	4.1614
CLOUD_COVERAGE	0.9143
TEMPERATURE	5.5696
WIND_SPEED	2.1772
PRECIPITATION	4.1750
DISTANCE_KM	22.9618
RUSH_HOUR_BINARY	-1.3029
WEEKEND_BINARY	-0.8003

Lasso Regression Coefficients (Selected Features)

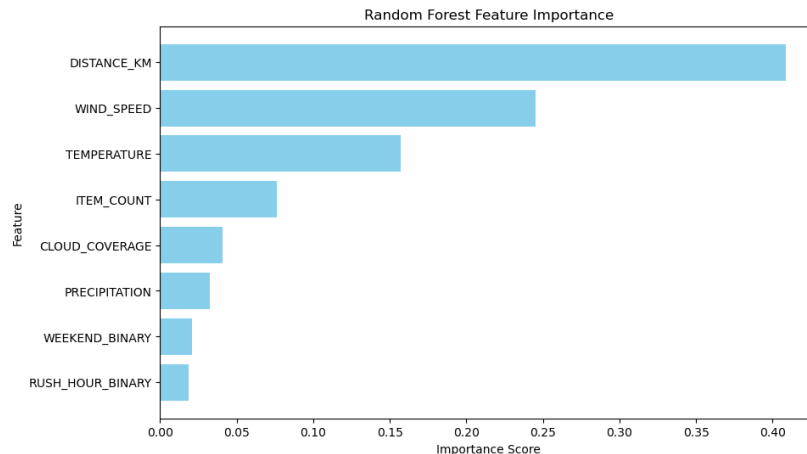
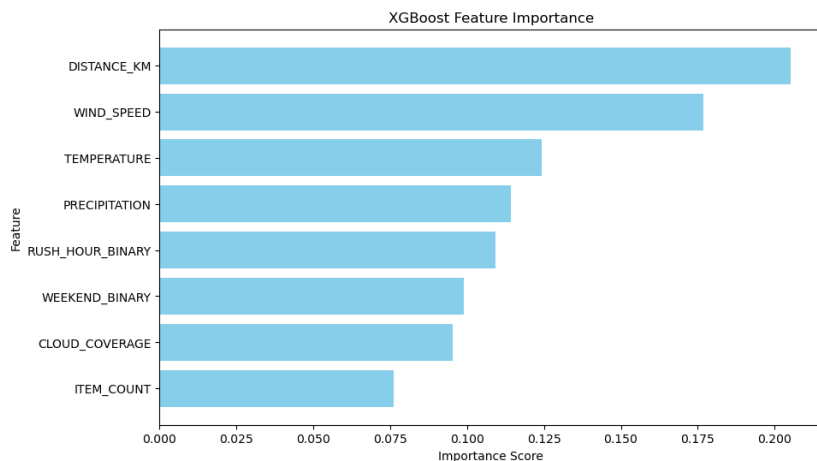


Ridge Regression Coefficients (Selected Features)



4. Methods (5/7) – Features

- **XGBoost and Random Forest feature importance**
 - We can see that their performance relates heavily on the distance.
 - However, XGBoost seems to find more of the less important features in Random Forest
 - For example, precipitation and cloud coverage.
 - Furthermore, we can see that item_count seems to have the 4th highest importance in Random Forest but in XGBoost it is the least important feature.



4. Methods (6/7) – Hyperparameter tuning

- We did hyperparameter tuning for our models in order to have them optimized (i.e. get the most out of them).
- We applied GridSearchCV and RandomizedSearchCV.
 - Since Linear Regression does not have meaningful hyperparameters to tune, it was left out.
 - **GridSearchCV** was used: XGBoost, Lasso - and Ridge regression.
 - Hyperparameters can drastically change performance and fine-tuning them is essential for achieving high accuracy.
 - By using cross-validation within GridSearchCV, we are ensuring that the selected hyperparameters generalize well across different parts of the data, leading to more reliable predictions on new, unseen data.
 - Manually tuning hyperparameters is difficult and prone to error. GridSearchCV provides a systematic, automated way to find the best settings, saving time and ensuring a thorough exploration of the hyperparameter space.
 - **RandomizedSearchCV** was used for the Random Forest model due to an error while using GridSearchCV for the Random Forest
 - RandomizedSearchCV was used since it randomly samples a subset of combinations, which can significantly reduce the computational burden.

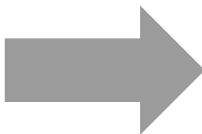
```
Best Random Forest Model from RandomizedSearchCV: RandomForestRegressor(max_depth=20, max_features='sqrt', min_samples_leaf=2,
min_samples_split=10, n_estimators=300,
random_state=1234)
```

4. Methods (7/7) – Trying categorical target variable

After having the preliminary results for our analysis based on continuous target variable, we decided to check briefly if the categorical variable gave as better results.

We used XGBoost because it turned out to be the best-performing model (see the final results on the next slide.)

Unfortunately, these results are not very good either.



Classification Report:

	precision	recall	f1-score	support
0	0.58	0.68	0.62	1643
1	0.52	0.55	0.53	1581
2	0.37	0.06	0.11	462
accuracy			0.55	3686
macro avg	0.49	0.43	0.42	3686
weighted avg	0.53	0.55	0.52	3686

Key Insights

Class Performance:

- The model performs relatively well for class 0 (0-30 mins) but struggles significantly with class 2 (45+ mins), with very low recall and F1-score. This indicates that many instances that should be classified as class 2 are likely misclassified as class 0 or class 1.

Class Imbalance:

- The distribution of instances among the classes may affect model performance. If class 0 has significantly more instances than class 2, the model might be biased towards predicting class 0.

Overall Model Effectiveness:

- An accuracy of about 54.58% suggests the model may not yet be robust enough for practical use, particularly in applications requiring reliable predictions across all categories.

5. Results (1/3)

Findings Overview

- From a data perspective, it is clear that no model achieves exceptional accuracy.
- This highlights the complexity of forecasting weather-based delivery times.
- Lower values of the R-quantile, all below 0.2, indicate that weather-related variables explain only a small fraction of the variation in delivery times.
- Of the models tested, Tuned XGBoost performs best when considering its root mean square error (RMSE) of 8.98 and R-squared value of 0.1927.
- This suggests that while all models struggle to predict with high accuracy, XGBoost's ability to handle complex models offers some improvement over simpler regression models such as Ridge or Lasso.
- The relatively high RMSE and low R-squared of all models indicate that the prediction of delivery times is relatively poor. This suggests that factors other than weather conditions, for example traffic patterns or restaurant-specific delays, may play a more important role in the variation in delivery times.

Comparing findings

- Our results (XGBoost) are similar to Wolt's (based on *estimated_delivery minutes*).
- This confirms the conclusion that weather conditions alone are not sufficient to accurately predict delivery times.

	WOLT	Group 25
MAE:	7.2537	7.2441
RMSE:	9.0609	8.9820
R^2	0.1824	0.1927

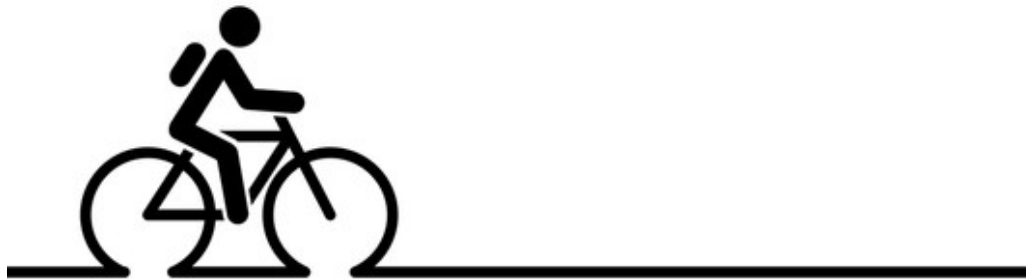
Model	Mean Absolute Error (MAE)	Root Mean Squared Error (RMSE)	R-squared (R²)
Ridge Regression	7.72544	9.4453	0.107278
Lasso Regression	7.74113	9.46427	0.103687
Tuned XGBoost	7.24512	8.98196	0.192714
Linear Regression	7.72484	9.44523	0.107291
Random Forest	7.40181	9.12617	0.166584

5. Discussion (2/3)

- Although weather conditions affect delivery times, their impact seems to be marginal.
 - Our models have consistently low R-squared values, indicating that weather-related variables explain only a small fraction of the variation in delivery times.
 - Prevailing forecast factors, such as distance and traffic conditions, play a much larger role in delaying delivery times.
 - This suggests that focusing on weather alone would not significantly improve the accuracy of delivery time forecasts. Instead, weather should be seen as a secondary factor that is best used in conjunction with other operational variables to refine forecast models.
- However, for Wolt to analyze the effectiveness of weather more thoroughly, they could add additional features to the models.
 - These features could be things such as the amount of couriers working at a given hour and how many orders there are made.
 - This could enhance the model since it does not include supply and demand.
 - Could poor weather lower the interest of customers in ordering food? Could the opposite happen?

5. Business recommendation (3/3)

- Wolt should focus on optimizing distance and operational factors based on our analysis. Since the weather is not a key driver of delays, efforts should focus on things such as route optimization, rush hour traffic management and improving restaurant preparation times. Tackling these factors could deliver more business value and customer satisfaction than relying on weather information.



6. Conclusion

- Delivery time influences can be searched using different machine learning methods.
- In our analysis of Wolt, we have used 5 different ML methods: Linear Regression, Ridge Regression, Lasso Regression, Random Forest and XGBoost.
 - ...to compute the dependencies of different features to delivery times.
- In Wolt operations, the weather has a marginal impact on delivery times, but it is not a major factor.
 - The low R^2 values indicate that there are other metrics that might have a bigger impact.
 - Although variables like rain and air temperature have a minor influence, they are not as critical as initially assumed.
 - Distance is by far the most important factor, and traffic patterns (e.g. peak periods) also play a more significant role in determining delivery delays.



References:

[1] Wolt. (2024, September 26). Wikipedia. [Accessed October 19, 2024].

Available: <https://en.wikipedia.org/wiki/Wolt>

[2] About Wolt. (2024). Wolt. [Accessed October 19, 2024]. Available:

<https://explore.wolt.com/fi/fin/about>

[3] Pussinen, J. (2024, February 1). Data Science Summer Intern assignment 2024. Github. [Accessed October 19, 2024]. Available:

<https://github.com/woltapp/data-science-internship-2024/tree/main>